

Caleb Lewis

COMP 2210

Mr. Seals

Graphs in Games

I. Introduction

In this project, I explored how a graph-based data structure can be used to represent territory-control games, specifically a simplified version of the popular board game Risk. Notably, Risk is a model world of connected regions, making it ideal for an exploration of games implemented as graph structures. As part of this project, I also explored the implementation of simple AI Agents as participants using simple heuristic algorithms. My interest in this project stemmed from game design using Swift to design iOS apps; however, in the grand scheme of things, my knowledge in game design is limited, and I wanted to expand my knowledge. To expand on my coursework in COMP 2210 Fundamentals of Computing II, I integrated key data structures that we talked about in class. Of these, the main three that I used were: Graphs, Maps, and Sets. Being able to tie in key course concepts to personal interests made Risk an ideal choice for this project. My goal for this project was neither to build a perfect AI nor to perfectly implement a Graph, but to learn about the process of implementing both. This paper discusses background ideas that influenced my design choices: multi-agent

reinforcement learning and game-playing agents. Next, it touches on the structure of my implementation, the heuristics used by my agents, and the challenges and observations I encountered along the way. Finally, it presents my experimental results from both Human vs Agent and Agent vs Agent tests and a reflection on what this project taught me about heuristics, graphs, and software design as a whole.

II. Background

i. Risk as a Graph

The core mechanic behind Risk relies on adjacency. Moving troops between neighboring countries and conquering the countries on your borders naturally lends itself to a graph structure with edges representing valid movements and nodes as the countries themselves. Risk's original rule set dictates reinforcement troops, attacks of neighboring countries, and fortification of owned countries; all of these hinge on evaluating relationships between nodes, which sets up our ideal environment for a graph model.

ii. Simple Heuristics

This project focuses solely on a simple heuristic agent that makes its decisions using local information (e.g., troop count and adjacent countries). Unlike a reinforcement learning system, this approach mirrors basic AI implementations like many early games that use a rule-based strategy rather than any learned policy.

iii. Related Work

As part of this project, I read two academic papers related to this topic. The first, A Comprehensive Review of Multi-Agent Reinforcement Learning in Video Games (Hu et al., 2023), talks about key concepts and challenges in multi-agent environments. The second, Games for Artificial Intelligence Research: A Review and Perspectives (Yannakakis & Togelius, 2023), explores the dual role of AI in games: designing and playing. In conjunction, the papers gave me a broader view of behavior and the limitations of my agents. While I did not implement any deep learning agents, learning about them reinforced my decision to use simple heuristics for this project. Hu et al. (2023) proposed five issues of multi-agent learning, two of which are relevant to my project: the complexity of interactions increases exponentially with each agent, and sparse rewards make it difficult to attribute success to specific actions. Risk as a game can have many players interacting, increasing the complexity, and has many moves before somebody wins the game. Yannakakis & Togelius (2023) presented the dual role of games in AI research: as a creative domain for AI to help design and as a testbed for decision-making agents. Additionally, Yannakakis & Togelius (2023) concluded that predictable, human-like behavior is often more desirable than the best performance possible out of game-playing AI. Together, these papers supported my choice of a light heuristic for this project and helped me observe limitations of my agent during my experiments.

III. Methodology

i. Environment Architecture

My implementation of Risk is centered around a few core classes, each responsible for a different component of the game. First, I have a Country class that represents territories and is where information such as owner, troop count, and neighboring countries is stored. Second, the Risk_Map class functions as my graph structure, which maintains the set of countries, maps names to nodes, and establishes the adjacencies as defined by a physical Risk board. The gameplay is run through the Risk class, which handles turns, user input, and calls to my agent. A visual representation of the board is then handled separately through the MapPanel and RiskGUI classes. The Player class is how the user interacts with the game and works through the terminal to input moves. The Agent works through the ComputerPlayer class, which extends the Player class and modifies the turn functions with the heuristics detailed later.

ii. Implementation of the Graph

The nodes of the graph are represented by instances of the Country class, one for each territory on a standard Risk board. Adjacency relationships are explicitly established in the Risk_Map constructor, as bidirectional edges connecting two instances of the Country class. These are stored as a PIV in the Country instance as a Set of adjacent neighbors. Keeping track of these adjacencies is what ensures valid attacks and movements. A Set is used to store countries in an effort to avoid duplicates and allow for an efficient iteration over the map. A Map with keys of type String and Values of type Country is used to quickly look up a country by name during user input processing.

iii. Game Flow and Turns

The flow of this implementation follows the traditional Risk sequence of moves: reinforcement, attack, and fortification. During reinforcement phases, the acting player is allocated new troops to place on controlled territories. Attack phases consist of an acting player selecting to use placed troops in a controlled territory to try and conquer a neighboring uncontrolled territory. This follows the traditional Risk rules: the attacker may attack with as many as $N-1$ troops (rolling 3 dice maximum), where N is the number of troops in the attacking country; the defender must defend with the number of troops in the defending country, then dice are rolled and resolved according to the rules of Risk combat. When a territory is conquered, attacking troops are able to be moved into the acquired territory, and ownership is updated.

iv. Agent Design

My agent is designed with and acts through a series of heuristics to determine moves. First, the agent uses a border-defense heuristic. All available (unplaced) troops are placed on the weakest border country: a country bordering at least one enemy territory and with the fewest placed troops. In the event that it does not control a border country, reinforcements are placed on an arbitrarily owned country. Next, the agent evaluates the countries that it has control of and identifies potential attackers. It defines a valid attacker as a country that has at least 2 more troops than an uncontrolled neighbor, prioritizing those with a greater advantage. When no valid attackers are found, it ends the attack phase. When it successfully defeats a country, it moves all but one of the troops from the attacking territory to the new territory, which aggressively moves its front line forward. Because of the local scope of the

Fig. 1, A Screenshot of the GUI visual showing troop counts overlaid onto a world map background.

vi. Challenges

My greatest challenge was realizing how I needed to build from node objects up, versus trying to start with a graph immediately. Initially, I started developing a Graph class that I would use to implement Risk, but I realized that once I started trying to play the game, it was difficult to get the data I needed from my implementation. I then started work on the current implementation as defined previously. Another challenge was a time constraint on the Agent functionality. Because of the slow start, steep learning curve, and failed first attempt, I was left with less time than anticipated to work on my Agent.

IV. Experiments

i. Setup

To evaluate my Agent, I conducted both Human vs Agent and Agent vs Agent games. Each experiment was run multiple times with these conditions: random assignment of countries, equal initial troop counts, and identical map configurations. All experiments were run using the terminal, while the GUI provided a visual of troop and territory changes. For the Human vs Agent, I played several incomplete games against the Agent and one complete game. For Agent vs Agent, I ran a series of a couple thousand tests and collected some interesting results noted later.

ii. Agent vs Agent

I conducted two distinct Agent vs Agent tests: the first with the whole map randomly assigned ownership, and the second with each agent randomly assigned one territory. From these tests, a few distinct patterns emerged. In the first, whichever agent controls a slightly larger contiguous area at the start snowballs early. Reinforcements converge on one border territory early, and it becomes a megastack: a country with a massively disproportionate number of troops. For example, Northern Europe is repeatedly chosen as the weakest border early on; consequently, there was a game in which Northern Europe accumulated over 133,000 troops. However, it often becomes unreachable in the late game because it gets isolated, and the heuristics ignore it because it is no longer weak nor a border country. When this happens, victory is stalled despite an overwhelming advantage because the megastack borders no opposing territories. Additionally, the losing player is almost always reduced to Western Australia as its last territory. In the second test, there was much of the same. Player 1 almost always forms a megastack in Northern Europe, and player 2 in Western Australia. The entire game tends to be reduced to a single contest over Indonesia in which Player 1 is attacking from New Guinea and Player 2 is attacking from Australia. Overall, I noticed across both tests that the agent consistently waited for an advantage and then aggressively attacked until there was no longer an advantage. The “weakest border” heuristic consistently concentrated troops into small dominant areas. The agent lacks long-term planning and never recognizes potential wins or transitions to defense once “winning”. Overall, I noticed that Player 1 would win the majority of the time, likely due to being able to attack first and control a larger contiguous area, leading to a victory down the line. In test 1,

Player 1 would consistently win 70-80% of the time, while in test 2, this dropped to about 60% of the time.

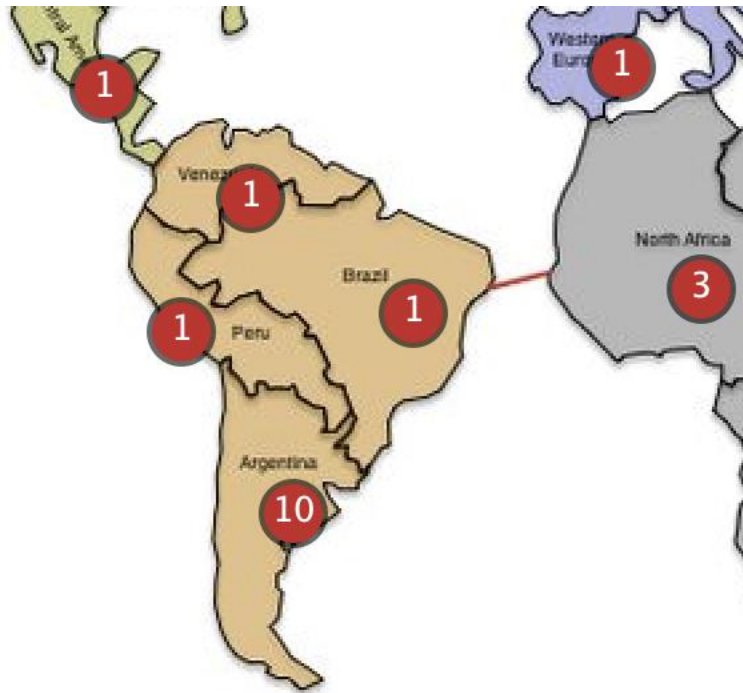


Fig. 2, GUI view of reinforcements being stacked onto one country without much use of them.

```
--- Computer Player 1 turn ---
Computer placed 11 troops on Eastern United States
Computer attacks Central America from Eastern United States
Roll result: attacker loses 1, defender loses 0.
Eastern United States now has 11 troops; Central America now has 1 troops.
Roll result: attacker loses 0, defender loses 1.
Eastern United States now has 11 troops; Central America now has 0 troops.
Computer conquers Central America!
Computer attacks Western United States from Central America
Roll result: attacker loses 0, defender loses 1.
Central America now has 10 troops; Western United States now has 0 troops.
Computer conquers Western United States!
Computer attacks Alberta from Western United States
Roll result: attacker loses 1, defender loses 0.
Western United States now has 8 troops; Alberta now has 1 troops.
Roll result: attacker loses 1, defender loses 0.
Western United States now has 7 troops; Alberta now has 1 troops.
Roll result: attacker loses 0, defender loses 1.
Western United States now has 7 troops; Alberta now has 0 troops.
Computer conquers Alberta!
Computer attacks Northwest Territory from Alberta
Roll result: attacker loses 1, defender loses 1.
Alberta now has 5 troops; Northwest Territory now has 1 troops.
Roll result: attacker loses 0, defender loses 1.
Alberta now has 5 troops; Northwest Territory now has 0 troops.
Computer conquers Northwest Territory!
Computer fortifies North Africa from East Africa with 1 troops.
Computer Player 1 ends turn.
```

Fig. 3, Excerpt from terminal showing the aggressive conquest that the Agent will go on.

iii. Human vs Agent

While playing against the agent, the early game feels fast and unpredictable due to the Agent instantly executing moves, often conquering multiple territories as it goes. However, after a few turns, it becomes obvious that it is leaving no defense behind, as shown by the trail of one troop country it leaves in its wake. The lack of defensive coordination allows for easy (although time-consuming) victories for the human. By spreading out troops and making a good defensible position, I was able to slow down the momentum of the Agent and exploit its weak sides. By forcing it to overextend its resources, human players are reliably able to defeat the Agent without much effort.

V. Conclusions

i. Findings

The experiments exposed the local heuristics working in the background of the Agent. They produced an aggressive early game expansion but lacked long-term stability. Although the idea of strengthening the weakest border sounds solid, in this implementation, it can lead to several unintended imbalances, most notably 133,000 troops in Northern Europe. These experiments proved that a lack of long-term plans causes collapse in the form of cyclic interactions or stalemates. In the future, I would focus on expanding the awareness that the agent has, letting it observe beyond immediate neighbors. This would enable it to evaluate threats and explore

opportunities requiring multiple turns. Introducing a defensive heuristic, like territory valuation, would help prevent the Agent from overextending, as was present in the experimental data.

ii. What I Learned

Throughout this project, I was able to better understand the implementation of graphs in a real-world game environment. I learned about how adjacency fundamentally shapes how the game unfolds. Implementing Agents expanded my appreciation of the gap between simple heuristics and sophisticated strategy and reasoning. I was able to strengthen my knowledge of Java and object-oriented programming. This project helped me to understand the fundamentals of Graphs in gameplay.

References

Li, Zhengyang, et al. "A comprehensive review of multi-agent reinforcement learning in video games." *IEEE Transactions on Games* (2025).

Hu, Chengpeng, et al. "Games for artificial intelligence research: A review and perspectives." *IEEE Transactions on Artificial Intelligence* 5.12 (2024): 5949-5968.